

Experiment → 7

There are n cities connected by some number of flights. you are given an array flights where $\text{flight}[i] = [\text{from}, \text{to}, \text{price}]$ indicates that there is a flight from city to city to, with cost price.

You are also given three integers src , dst and K , return the cheapest price from src to dst with at most K stops. If there is no such route, return -1 .

Input $\rightarrow n=4$, $\text{flights} = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]$, $\text{src} = 0$, $\text{dst} = 3$, $K = 1$.

Solⁿ: Since we have a constraint on K stops, normal Dijkstra doesn't work. So, we will use Bellman Ford ($K+1$ relaxations).

Algorithm :

① Initialize $\text{dist}[i] = \infty$ for all cities.
 $\text{dist}[\text{src}] = 0$.

② Repeat $K+1$ times

Copy $\text{dist} \rightarrow \text{temp}$ (This extra array is used to prevent using values from same iteration)
For every flight $[u, v, \text{price}]$
if $\text{dist}[u] + \text{price} < \text{temp}[v] \rightarrow \text{update temp}[v]$.

③ Update $\text{dist} = \text{temp}$

④ After $K+1$ iterations

if $\text{dist}[\text{dst}] == \infty \rightarrow \text{return } -1$
else return $\text{dist}[\text{dst}]$.

Time Complexity $\rightarrow O(K \times E)$, Space Complexity $\rightarrow O(N)$ $\hookrightarrow$ no. of cities

→ Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
int findCheapestPrice(int n, vector<vector<int>>& flights,
                     int src, int dst, int k) {
```

```
    vector<int> dist(n, INT_MAX);
    dist[src] = 0;
```

```
    // Relax edges k+1 times
```

```
    for (int i = 0; i <= k; i++) {
```

```
        vector<int> temp = dist;
```

```
        for (auto & flight : flights) {
```

```
            int u = flight[0];
```

```
            int v = flight[1];
```

```
            int price = flight[2];
```

```
            if (dist[u] != INT_MAX && dist[u] + price < temp[v])
```

```
                temp[v] = dist[u] + price;
```

```
        }
```

```
    }
```

```
    dist = temp;
```

```
    return dist[dst] == INT_MAX ? -1 : dist[dst];
```

```
}
```

```
}
```